Análisis léxico y sintáctico:
Caso de estudio para la lógica proposicional

Martí Pons Garcia

Marzo de 2026

Resumen

Este artículo presenta el desarrollo de un analizador léxico y sintáctico aplicado al lenguaje de la lógica proposicional. Mediante el uso del lenguaje de programación funcional Haskell, se implementan las bases teóricas de autómatas y gramáticas, mostrando su aplicación en la construcción de un lexer y un parser. El documento detalla la identificación de tokens (lexemas), la construcción de árboles de sintaxis abstracta (AST) usando gramáticas LL(1) y combinadores de parsers. Culmina con la evaluación semántica de fórmulas para testar tautologías.

Índice

1. Introducción **2**

2. Análisis léxico **2**
2.1. La noción de *token* y regla léxica 2
2.2. Reconocimiento de **tokens** 3

3. Análisis sintáctico **5**
3.1. Del árbol de derivación al AST 5
3.2. El parser como tipo abstracto: combinadores y descenso recursivo 7

4. Caso de estudio: lógica proposicional **10**
4.1. Tokens de la lógica y analizador léxico 11
4.2. La gramática LL(1) para la lógica, el AST y el analizador sintáctico 12
4.3. Evaluación semántica 16

5. Conclusiones **17**

1. Introducción

Este artículo complementa el estudio de formalización presentado en el trabajo Pons Garcia. Se toman los modelos computacionales allí descritos, presentados como objetos matemáticos, para construir lo que constituiría las primeras fases del análisis de texto presentes en los compiladores.

En primer lugar, mostraremos el rol que juegan dichos modelos en el proceso de compilación de un lenguaje formal. Crearemos las herramientas basadas en autómatas y gramáticas que nos permiten llevar a cabo un *análisis léxico* y posteriormente un *análisis sintáctico* de un lenguaje formal. Veremos de nuevo cómo la implementación en Haskell nos permite definir las estructuras para que se comporten como nos interesa.

Finalmente, para aterrizar las formalizaciones abstractas que hemos definido para analizar textos, tomaremos un lenguaje formal particular: el lenguaje de la lógica proposicional. Este nos permitirá ilustrar cómo actúa cada uno de los bloques destinados al análisis. Además, al ser un lenguaje decidible, podremos añadir una capa semántica que se basará en la interpretación clásica y nos permitirá evaluar tautologías.

2. Análisis léxico

En la teoría de la computación, los compiladores conforman una parte esencial para poder comunicarnos con las máquinas a través de nuestros lenguajes formales, generalmente lenguajes de programación. Para ello existen las primeras fases de análisis, en las que se procesa un texto para detallar sus partes. La primera parte de este proceso es el análisis léxico, que se encarga de reconocer las unidades básicas de nuestro lenguaje.

2.1. La noción de *token* y regla léxica

Cuando uno se pregunta por los átomos de un lenguaje, las unidades inseparables, en realidad no se piensa en caracteres. Esto se debe a que unidad básica se refiere a mínima unidad con significado. Esto, en lenguaje natural, se podría traducir a las palabras. Para nosotros, estos elementos van a ser los *tokens*. Entonces, cuando queramos definir un lenguaje, en primer lugar definiremos los tokens que usaremos. Veremos un ejemplo de esto en la sección 4.1, con el caso de estudio concreto.

Ahora que tenemos sobre la mesa la idea de un token, pensemos en cómo actuará nuestro analizador léxico, que llamaremos de ahora en adelante **lexer**. Lo que tiene que hacer esta función será tomar una cadena e identificar en ella los posibles tokens que hay allí. De esta manera, definiremos un ADT (recordemos, *Abstract Data Type*) con el que reconocer las palabras de nuestro texto.

Ahora el punto está en cómo reconocer estos tokens. Dado que las palabras son cadenas concretas, cada uno de estos tokens se puede ver como un lenguaje regular. La idea es, entonces, utilizar los DFA para reconocer estas posibles palabras. Crearemos para cada token una regla léxica que asocia un DFA a una función de conversión. La regla léxica que se muestra en el código 1 nos sirve para decir, de una cadena *st*, que si esta es aceptada por el autómata DFA, entonces emplee la función que convierte esa cadena al tipo token.

Pensemos en esta conversión como la acción que hace un niño que está aprendiendo a leer. Este va recitando en voz alta las letras de la palabra, está analizando los caracteres. Cuando termina,

pronuncia la palabra entera de golpe. En ese momento, la palabra ya no se trata de una cadena de letras. En su lugar es un bloque léxico entero que denominamos palabra. Ha convertido el tipo «cadena de caracteres» en el tipo «palabra». El token.

Listing 1: LexRule asocia un autómata a una función que crea un token desde el string aceptado.

```
type LexRule token st = (DFA st, String -> token)
```

Para un texto no tendremos una única regla léxica, sino un conjunto de reglas, una para cada token[1]. Esto significa que tenemos que nuestro lexer va a tener en cuenta todos los posibles tokens a medida que vaya leyendo la cadena.

2.2. Reconocimiento de tokens

Para procesar una cadena de texto y convertirla en tokens, definiremos una función `runLexer` que tome el texto y una lista de reglas léxicas, y nos devuelva, o bien un error léxico (sería como una falta de ortografía) o bien la lista ordenada de los tokens que conforman el texto.

Antes de presentar el algoritmo para esta función, hay que tener en cuenta dos casuísticas que se pueden dar en el procesamiento del texto:

- Por un lado, puede haber dos tokens que acepten los mismos caracteres hasta cierto punto. Pongamos por ejemplo las palabras 'orden' y 'ordenador'. Fijémonos en que las primeras cinco letras son las mismas. De esta manera, nuestro lexer está en la misma disposición de escoger una u otra.
- De la misma forma que puede haber coincidencia en un cierto número de caracteres, también puede haber dos palabras iguales que puedan ser aceptadas por más de una regla léxica. Este es el caso de los tokens que nos describen unidades de significado concreto. Por ejemplo, si estamos en un lenguaje de programación, una variable se puede denotar generalmente por una cadena de letras (el token *Var*). Pero también hay casos en los que nos interesa denotar que un objeto se trata de un número entero. En este caso usamos la cadena 'int' (el token *Entero*). Vemos que hay una misma posible cadena para dos tokens distintos.

Estos casos de ambigüedad se resuelven imponiendo dos reglas estandarizadas. En primer lugar, para el primer caso se usa la regla de escoger la cadena más larga que presente token. Vemos que esto nos obliga a que los tokens sean siempre cadenas con una longitud limitada, lo cual es obvio en un contexto aplicativo como en el que estamos. Por otro lado, la segunda casuística se resuelve simplemente dando un orden de prioridad a las reglas léxicas.

Vamos a ver cómo actúa nuestra función `longestMatch` que busca en un texto el primer token, respetando las reglas de búsqueda que hemos definido. Partimos de una lista de reglas *rules* y un texto *input*.

- Primero de todo, calculemos la lista de las coincidencias, llamada `matches`. Es decir, la lista de tokens que han reconocido cada una de nuestras reglas léxicas. Es posible que solo apliquen un subconjunto de estas. Este reconocimiento lo hacemos a través de una función, `matchDFA`, que se dedica a mirar en una cadena si hay una subcadena (lo más larga posible) que sea aceptada por un DFA dado.

[1] Pensemos que un token no es exactamente una palabra, sino una unidad léxica. Esto significa que puede haber una o más cadenas que tengan el mismo significado. Por ejemplo, si quisiéramos analizar una lengua, no puede venir bien poner en el mismo token, llamémoslo *artículo*, a las cadenas 'la' y 'el'.

- Una vez calculados los `matches` hacemos recursión sobre la lista. En caso de estar vacía, devolvemos un aviso (`Nothing`) para que el lexer sepa que ha habido un error, pues esto significa que la cadena no tiene ningún token y, por lo tanto, está mal formada. En otro caso, aplicamos una función `findBest` sobre el primer elemento de la lista para buscar el mejor.

- Recordemos que el criterio "el mejor" consiste en buscar la cadena más larga y, en caso de igual longitud, mirar la preferencia de reglas. Entonces, `findBest` compara la longitud del primer elemento de la lista con el segundo y se queda con el token que provenga de la cadena más larga. En caso de empate, se queda con el primero, pues la preferencia viene dada por el orden de la lista de reglas.

- Cuando la recursión llega a que no quedan más elementos de `matches` para comparar, se devuelva el token "ganador", junto con el resto de la cadena (la parte que no forma parte del token).

Una vez que tenemos la manera de leer el primer token de un texto, podemos crear nuestra función `runLexer`, que hace el análisis completo de la cadena, para obtener la lista de tokens. Partimos, como antes, de una lista de reglas *rules* y un texto *input*.

Listing 2: La función que implementa el lexer utiliza la lista de reglas léxicas para leer un texto y devolver la lista de tokens, o un mensaje de error con el índice en el que se ha producido el fallo.

```
runLexer :: [LexRule token st] -> String -> Either LexError [token]
```

Utilizaremos un acumulador para guardar los tokens que vamos descubriendo. Así, aplicamos iterativamente `longestMatch` y vamos guardando el token resultante en el acumulador. Luego, aplicaremos de nuevo la función, pero sobre la nueva cadena que hemos recortado, sacando la subcadena del token guardado. Este proceso iterativo se termina cuando no hay más caracteres para leer, y solo hemos de tener en cuenta que el acumulador tiene la lista de tokens al revés.

Un tratamiento interesante que le damos a la función del lexer es que nos permite controlar el error léxico. En cada momento que aplicamos `longestMatch` hay la posibilidad de error en el próximo token. Usando la longitud de la cadena procesada hasta el momento, podemos advertir, no solo que ha habido un error léxico, sino además la posición de la cadena en la que se ha producido. Ese control se hace usando un tipo, llamado `LexError` que podemos encontrar en el código 2, formado por la posición mencionada y un mensaje de error.

Con esta construcción completamente genérica (pues solo depende de la lista de reglas léxicas del lenguaje), que fundamentalmente se basa en autómatas finitos, ya tenemos un método para reconocer las "palabras" de nuestro lenguaje formal.

3. Análisis sintáctico

Una vez tenemos el conjunto de tokens de nuestro texto, hay que tener en cuenta que no se admite cualquier secuencia. No vamos a acceptar "ordenador el", pero sí aceptaremos "el ordenador". En la analogía con un lenguaje, queremos pasar a analizar las oraciones, cuyas unidades básicas son las palabras que ya tenemos procesadas. Este procesamiento exige verificar la formación de la cadena de tokens. Además, la complejidad de las construcciones hará que los lenguajes que trabajemos (formados por cadenas de tokens) sean de tipo independiente del contexto.

3.1. Del árbol de derivación al AST

Para proceder al análisis sintáctico, hemos de hallar la estructura que nos permita expresar esa construcción. Sea $G = \langle V, T, P, S \rangle$ una CFG, el **árbol de derivación** de G nos permite expresar la construcción de una cadena $w \in L(G)$ a partir del símbolo inicial.

Definición 1. ***(Árbol de derivación)***
Un árbol es de derivación si cumple las condiciones siguientes:

- *Cada nodo interior está etiquetado con un símbolo no terminal $n \in V$.*
- *Cada hoja está etiquetada con un símbolo terminal $t \in T$.*
- *De cada nodo se desprende una lista de hijos (que puede ser vacía, simbolizando la producción $A \to \varepsilon$), que pueden ser nuevos nodos u hojas. Si la lista de hijos está etiquetada con*

$$X_1, X_2, \ldots, X_k$$

entonces, comenzando por la izquierda, $A \to X_1 X_2 \ldots X_k$ es una producción de P.

Podemos ver un ejemplo en la figura 1 para la cadena 0110. Observemos que aquí tenemos aplicadas las producciones $P \to 0P0$, $P \to 1P1$ y $P \to \varepsilon$. Este es un objeto que se puede codificar

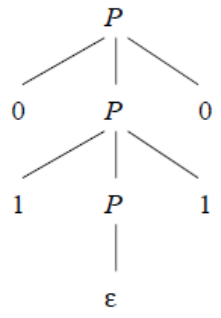

Figura 1: Árbol de derivación para $P \xrightarrow{*} 0110$.
Figura de Hopcroft, Motwani y Ullman, p. 156.

fácilmente como ADT, como podemos comprobar por el código 3.

Listing 3: Definición del tipo de dato DerTree. Este es, o bien una hoja, o bien un nodo con una lista de hijos.

```
data DerTree nt t = Leaf t | Node nt [DerTree nt t]
```

Vemos que, en un árbol completamente desplegado (es decir, cuando conocemos todos los valores de sus nodos), nos interesa poder recoger esta cadena final, pues las hojas son los terminales o ε que conforman las cadenas de nuestro lenguaje. Esta acción se puede realizar por medio de un proceso recursivo, encapsulado por la función `yield`, donde se exploren las ramificaciones del árbol, siempre partiendo de izquierda a derecha[2], hasta encontrar una hoja: un símbolo terminal, o la cadena vacía ε.

Pero lo que resulta interesante de estos tipos de datos es que nos permiten aplicar las derivaciones de una CFG, recordando todo el camino del que ha surgido la forma sentencial final. De esta manera, podemos implementar, por ejemplo, las funciones `deriveChild` y `derTreeStar`, que vemos en el código 4. Estas son funciones análogas a las `der1` y `derStar` que podríamos tener para las formas sentenciales simples, pero que, en lugar de generar un conjunto de cadenas, producen un conjunto de árboles de derivación. Aplicando la función `yield` sobre cada árbol que nos devuelve `derTreeStar`, recuperamos las formas sentenciales directas.

Listing 4: Las funciones deriveChild y derTreeStar generan los árboles que se pueden obtener con un paso de derivación y haciendo el cierre recursivo-transitivo, respectivamente.

```
deriveChild :: CFG nt t -> DerTree nt t -> S.Set (DerTree nt t)
derTreeStar :: CFG nt t -> DerTree nt t -> S.Set (DerTree nt t)
```

En este punto podemos visualizar una primera idea de cómo implementar nuestro parser. Con las funciones mencionadas, podríamos realizar, en primer lugar, una verificación de si la cadena de tokens pertenece al lenguaje (el reconocimiento podría llevarse a cabo mediante el PDA generado a partir de la CFG) y, en segundo lugar, en caso afirmativo, determinar qué árbol generado por `derTreeStar` es el que produce nuestra cadena de tokens.

Pero este método es profundamente ineficiente. Ya solo el uso de `derTreeStar` nos puede llevar a un cálculo sin fin. Para esto vamos a introducir una estructura parecida a los árboles de derivación más adecuada para el proceso de parseado, tal y como se usa habitualmente en los compiladores.

Una derivación es, en esencia, una secuencia de formas sentenciales. Sin embargo, lo que nos interesa realmente para el análisis sintáctico no es ese camino lineal de sustituciones. En su lugar, queremos obtener la estructura jerárquica que se ha ido construyendo a lo largo de él. Esto significa que no necesitamos saber cuántos pasos se han dado ni en qué orden aparecen los símbolos no terminales intermedios. Lo que nos interesa saber es qué operador conecta con qué subexpresiones, y cómo estas se anidan entre sí. Un árbol de derivación contiene toda la información del proceso de derivación, de forma que también cuenta con toda la información superflua. De esta manera, cuando hacemos el análisis sintáctico, aparte de verificar la correcta formación de la cadena, solo necesitamos retener la estructura gramatical relevante para el significado.

A diferencia del árbol de derivación, que encapsula el proceso entero de formación de la cadena, un **AST** (*Abstract Syntax Tree*) (Aho et al.) permite obtener solamente la información relevante. Un AST es un árbol más económico para el diseño de traductores que el árbol de derivación, donde los nodos interiores son operadores y sus hijos son los operandos. De esta manera, parsear "$1 + 3$" consiste en obtener el árbol con un nodo '+' y dos hijos '1' y '3'.

[2]De la misma forma que pasaba con las producciones de las gramáticas regulares en Pons Garcia, aquí también se podría dar la definión de derecha a izquierda.

El objetivo es, por lo tanto, construir un algoritmo eficiente dirigido por la entrada que, al reconocer cada construcción gramatical, genere el nodo del AST correspondiente. Todo ello, sin dejar de lado los conceptos fundamentales de gramáticas y autómatas.

3.2. El parser como tipo abstracto: combinadores y descenso recursivo

El análisis sintáctico que queremos implementar va a tener por objetivo tomar una lista de tokens sobre la que identificar la estructura jerárquica subyacente. Esta estructura nos la da el AST. La estrategia de parseado es de arriba a abajo (*top-down*) (Khan). Consiste en partir de los símbolos no terminales y analizar la cadena de tokens en el sentido de las producciones. Es decir, tratar de aplicar las diferentes producciones hasta llegar a los tokens que conforman nuestra cadena[3]. Si nos fijamos, la estrategia *top-down* es también la que planteamos para hacer el análisis con los árboles de derivación.

Para hacer la implementación sobre un AST, en lugar de tomar directamente una CFG, construiremos el algoritmo de análisis por partes, cada una dedicada a una de las producciones. Estas partes serán los elementos básicos de nuestra construcción. Serán elementos de un nuevo tipo que llamaremos `Parser`. Su función consistirá en "simular" las producciones de la gramática. Es decir, dada una cadena de tokens, tratará de consumirla según su regla y, en caso de que tenga éxito, nos devolverá el fragmento de AST construido junto al remanente de tokens que no ha consumido. En caso de que falle, nos devuelve un mensaje de error. Podemos ver la definición del tipo `Parser` en el código 5.

Listing 5: El tipo `Parser` consiste en una función que toma una lista de tokens, y devuelve, o bien un mensaje de error, o bien un par con la estructura del ADT y el resto de tokens que no ha consumido.

```
newtype Parser t a = Parser
  { runParser :: [t] -> Either String (a, [t]) }
```

De esta manera, el análisis sintáctico consistirá en un encadenamiento de elementos tipo `Parser`, que buscará consumir los tokens e ir construyendo en su lugar un AST completo.

Sin embargo, es importante precisar el rol de la gramática. Para hacer este proceso de consumir tokens de izquierda a derecha, de forma eficiente y predecible, tendremos que "simular" un tipo concreto de gramática independiente del contexto. Vamos a usar una gramática **LL(1)**[4]. Este es una clase de CFG específicamente diseñada para la construcción de analizadores sintácticos.

- La primera 'L' viene de *Left-to-right scanning* y se refiere a que el tipo de escaneo va de izquierda a derecha.
- La segunda 'L' se refiere a *Leftmost derivation*, que intenta producir primero la derivación por la izquierda. Es decir, primero trata de aplicar producción sobre el no terminal de más a la izquierda. Si la producción que busca implementar es $S \to AB$, primero se explorará el camino de A.

[3] La estrategia *top-down* contrasta con otra alternativa, la *bottom-up*, que toma los tokens de nuestra cadena para intentar predecir cáles son los no terminales que lo han generado (Khan).

[4] Un aspecto interesante que relaciona esta gramática con la teoría presentada hasta el momento es que los lenguajes generados por LL(1) son un subconjunto de los lenguajes aceptados por los autómatas de pila deterministas L_{DPDA} (Hopcroft, Motwani y Ullman).

- Finalmente, el 1 se refiere a que utiliza un único símbolo de entrada como preanálisis (*lookahead*) en cada paso para tomar las decisiones. Veremos que estas características se manifiestan en nuestra construcción del analizador.

En la práctica, para que una CFG sea LL(1) se deben cumplir las siguientes restricciones estructurales sobres sus producciones (Aho et al.):

1. **Ausencia de ambigüedad:** La gramática debe ser inequívoca, permitiendo una única derivación más a la izquierda para cualquier cadena válida.
   Un ejemplo clásico es $E \to E + E \mid E * E \mid \text{num}$. La cadena $1 + 2 * 3$ puede derivarse de dos maneras según qué producción se aplique primero, generando dos AST distintos, pues el orden de derivación condiciona la precedencia de los operadores.

2. **Ausencia de recursión por la izquierda (*Left Recursion*):** Una producción no puede empezar por el mismo no terminal que la define, ni directa ($A \to A\alpha$) ni indirectamente ($A \xrightarrow{k} A\alpha$), para evitar bucles infinitos en el análisis descendente.
   Pensemos en la producción $A \to A + B \mid B$. Al intentar expandir A, el analizador aplicaría $A \to A + B$... y nuevamente $A \to A + B$, sin consumir ningún token, entrando en un bucle infinito.

3. **Factorización por la izquierda (*Left Factoring*):** Si un no terminal tiene varias alternativas, estas no pueden empezar con una misma subsecuencia de símbolos ($A \to \alpha\beta \mid \alpha\gamma$), pues el analizador no podría decidir qué camino tomar con un solo símbolo de preanálisis.
   Si $A \to \texttt{if}\ E\ \texttt{then}\ S \mid \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S$, al ver el token '`if`', el analizador no sabe cuál de las dos alternativas tomar con un solo símbolo de lookahead.

4. **Determinismo en producciones vacías:** Si una alternativa de un no terminal puede derivar en la cadena vacía (ε), los símbolos que inician las demás alternativas deben ser disjuntos de aquellos que pueden aparecer inmediatamente después de dicho no terminal en el flujo de la entrada.

La primera restricción sirve para poder aplicar la *leftmost derivation* sin que ello nos provoque obtener más de un posible AST. Esta es una condición clave para que el proceso sea determinista. Por otro lado, la segunda característica nos interesa para evitar que el parser entre en un bucle infinito, pues si el primer símbolo no terminal es él mismo, probará de aplicar indefinidamente un parser que se remite a él mismo sin poder salir en ningún momento. Esto nos asegura que podemos escanear de izquierda a derecha (*Left-to-right scanning*) sin bloquearnos. Finalmente, las dos últimas condiciones (factorización por la izquierda y determinismo en producciones vacías) son las que aseguran que el *lookahead* de 1 símbolo sea siempre suficiente para tomar una decisión unívoca.

El método que usamos para parsear con una gramática LL(1), traduciendo cada no terminal a una función del tipo `Parser` se conoce como **descenso recursivo** (*recursive descent*) (Khan, p. 70).

Una vez establecido que cada símbolo no terminal de nuestra gramática se implementará como una función de tipo `Parser` nos enfrentamos al reto de cómo construir dicha función. Primero podemos empezar con algunos parsers que nos resultarán muy útiles: los analizadores atómicos `pFail`, `satisfy` y `eof`. Cada uno de ellos sirve, respectivamente, para: devolver el error, leer el primer token de entrada (para indicar si aplicar una u otra regla) y comprobar que se ha acabado el input.

Estos parsers atómicos parecen muy simples, pero definirlos independientemente orienta hacia la construcción que implementamos. Una producción de la gramática es mucho más compleja que un parser atómico, y definir el parser independientemente puede resultar muy poco práctico. En su lugar, usamos los **combinadores de parsers**. Un combinador es simplemente un operador matemático que toma uno o más analizadores pequeños y los junta para formar un analizador más complejo. De esta manera se puede encargar internamente de la identificación de tokens y la propagación de errores.

Un punto excelente en el uso de Haskell es que este lenguaje incorpora unas estructuras algebraicas estándar del lenguaje: las **clases de tipos** (*typeclass*). Estas nos proveen exactamente de las interfaces necesarias para ensamblar nuestros parsers (Hutton y Meijer). Pensemos en nuestro objeto matemático, el tipo `Parser`. Para poder utilizar estas interfaces para definir nuestros combinadores de parsers, hemos de probar que nuestro objeto pertenece a estas clases de tipos que nos interesan. Para ello, la tarea consistirá en definir las funciones propias de tales clases. En otras palabras, vamos a dar una prueba constructiva de que el tipo `Parser` cumple dichas propiedades. Veamos ahora las *typeclass* que nos interesan:

- La clase `Functor`. Esta clase es la que nos permite aplicar funciones dentro de un contenedor.
- Las clases `Applicative` y `Monad`. Son la clave de nuestra concatenación de parsers. Estas clases nos proveen de las operaciones '`<*>`' y '`>>=`'. La primera nos permite aplicar una función, ya dentro del contexto, a un valor que también se encuentra en ese contexto. Y la segunda, además, nos permite usar el resultado del primer parser en el siguiente. Esto es clave, porque la lista de tokens no leídos que nos devuelve un parser es en realidad lo que nos interesa utilizar en el segundo.
  Para ilustrar las operaciones de estas dos clases, usemos los siguientes ejemplos:

  ```
  do x <- [1, 2, 3]
     y <- [0, 1]
     [x * y]
  ```

- La clase `Alternative`. Para tratar con las ramificaciones que nos pueden proveer las producciones, necesitamos un método de *backtracking*. El operador `<|>` nos permite retroceder y, si el primer parser ha dado error, tratar de usar el segundo.

Con la definición de todas estas piezas abstractas, podemos dedicarnos a una implementación concreta de unas gramáticas, en la que solo tendremos que definir los parsers para cada no terminal e implementar el algoritmo general que los ensamble. Para ilustrar este procedimiento, dedicaremos la siguiente sección a un caso de estudio muy interesante en el que, además de poder dar el análisis sintáctico, también podremos añadir una semántica.

4. Caso de estudio: lógica proposicional

Para culminar la aplicación de los modelos estudiados a lo largo del trabajo, se presenta una implementación del análisis sintáctico del lenguaje de la lógica proposicional. Siguiendo la construcción acumulativa que se usa en los compiladores, trazaremos un recorrido como el que hemos presentado de forma abstracta en las secciones anteriores. En primer lugar, especificaremos el `lexer` de la lógica proposicional. Luego daremos una gramática para determinar la formación de fórmulas, y crearemos el `parser` correspondiente. Finalmente, para aprovechar el AST (*Abstract Syntax Tree*) generado por el parser, añadiremos una semántica a nuestro lenguaje. Esto es, asignar un valor de verdad a cada una de nuestras fórmulas bien formadas (aquellas que el parser acepta). En lógica proposicional siempre podemos recurrir a las tablas de verdad como método de evaluación. Esto nos garantiza tener un método de decisión para verifiacr siempre si una fórmula del lenguaje es una tautología (una fórmula válida). La figura 2 muestra el procedimiento descrito.

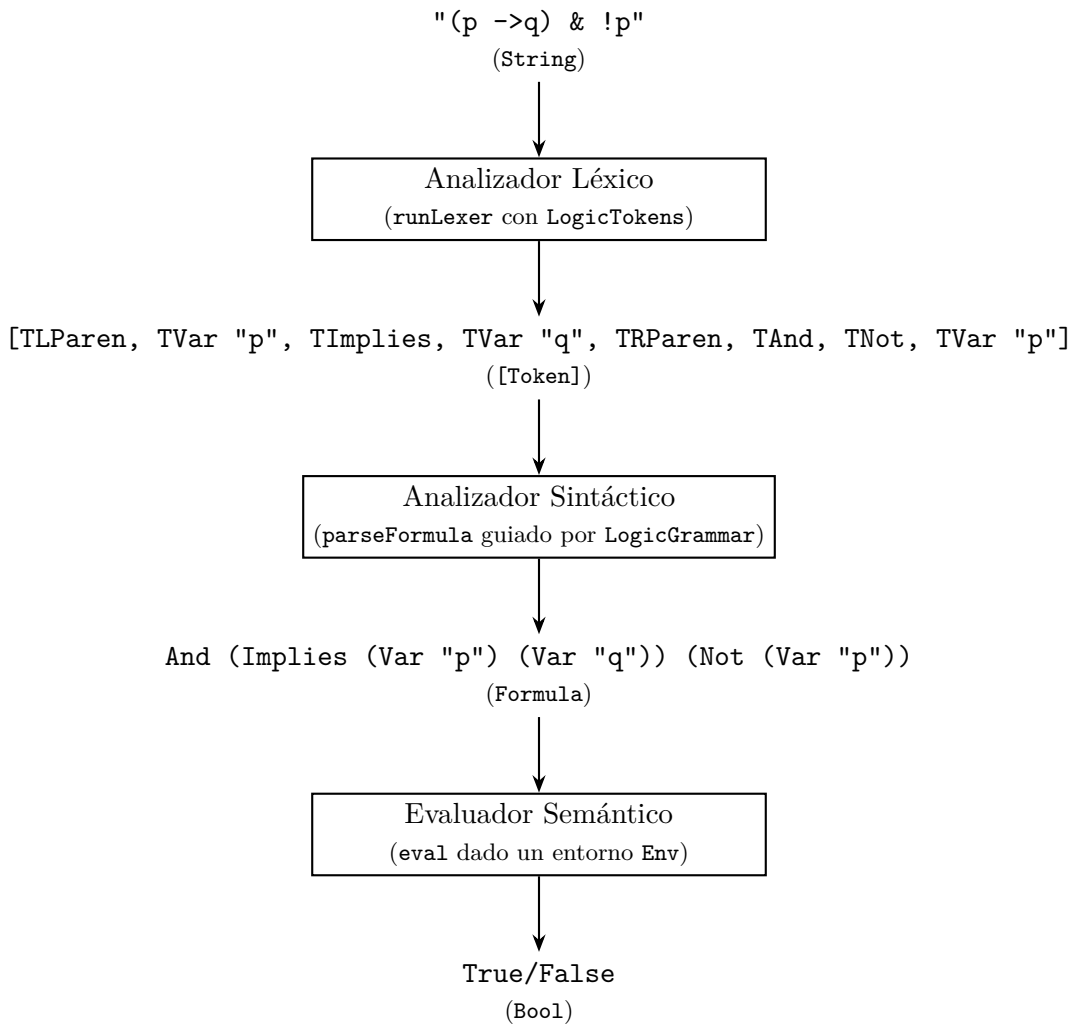

Figura 2: Diagrama de flujo del análisis implementado para la lógica proposicional. Los recuadros representan las fases de procesamiento y el texto intermedio ilustra cómo se transforma la información a través de los tipos de datos definidos (`String` → `[Token]` → `Formula` → `Bool`).

4.1. Tokens de la lógica y analizador léxico

Empecemos definiendo los tokens de la lógica. Normalmente hablamos del alfabeto del lenguaje, porque las unidades mínimas de la lógica son los símbolos. Pero vemos que es pertinente identificarlos como tokens, no solo para facilitar el análisis sintáctico posterior, sino también porque hay signos con más de un carácter.

Nuestro alfabeto, o conjunto de tokens, incluirá los conectores habituales: $\neg$, $\vee$, $\wedge$, $\rightarrow$ y $\leftrightarrow$. También tendremos las letras $p, q, r, \ldots, p_1, p_2, \ldots$ para las variables. También nos interesará poder identificar los paréntesis '(' y ')'.

Listing 6: Los tokens de nuestra lógica proposicional. Aparte de las variables, el resto de tokens corresponde a una o dos cadenas de carácters concretas.

```
data Token =
    TAnd          --  And o &
  | TOr           --  Or o |
  | TNot          --  ~ o !
  | TImplies      --  ->
  | TIff          --  <->
  | TLParen       --  (
  | TRParen       --  )
  | TVar String
```

El código 6 muestra los distintos tokens. Vemos comentadas las cadenas que podemos usar para referirnos a cada uno de estos tokens. Todos son estáticos, excepto las variables, que admiten múltiples cadenas, cada una de ellas representando un token distinto. Es decir, si escribimos el texto, «p Or rp2 | p1» el lexer deberá leer `[TVar 'p', TOr, TVar 'rp2', TOr, TVar 'p1']`. Es decir, tenemos cuatro tokens distintos, las variables `TVar 'p'`, `TVar 'rp2'` y `TVar 'p1'`, y un símbolo de disyunción `TOr` que se repite dos veces.

Para poder reconocer las cadenas, necesitamos definir un DFA para cada uno de los posibles tokens. Para ello nos resulta natural definir la gramática regular con la que generar dichas cadenas. Con nuestras funciones `rgToNFA` y `nfaToDFA` podemos fácilmente pasar de un modelo a otro. Veamos la gramática regular que usamos para identificar las variables $R_{Var} = \langle V_{Var}, T_{Var}, P_{Var}, S_{Var} \rangle$:

- $V_{Var} = \{0, 1\}$ (recordemos que no estamos limitados a usar letras para los símbolos no terminales).
- $T_{Var} = \{\text{'a'}, \text{'b'}, \ldots, \text{'z'}, \text{'0'}, \text{'1'}, \ldots, \text{'9'}\}$.
- $S_{Var} = \text{'0'}$.
- Las producciones de P_{Var} son
  - $0 \rightarrow c1 \mid c$, para $c \in \{\text{'a'}, \text{'b'}, \ldots, \text{'z'}\}$.
  - $1 \rightarrow c1 \mid c$, para $c \in \{\text{'a'}, \text{'b'}, \ldots, \text{'z'}, \text{'0'}, \text{'1'}, \ldots, \text{'9'}\}$.

Con esta definición, el "sublenguaje" de nuestras variables consistirá en las cadenas de letras minúsculas y dígitos que empiezan por una letra.

Para definir nuestro analizador sintáctico, solo tenemos que definir las reglas léxicas y aplicarlas a nuestro ADT del lexer. Es decir, vamos a definir una lista de pares (A_t, f_t) donde A_t es el DFA

que acepta las cadenas correspondientes al token t, y f_t es la función que toma una cadena (la que habrá identificado el autómata) y la convierte en el token t. Con esto, el analizador léxico queda definido como se muestra en el código 7.

Listing 7: Definición del lexer para la lógica proposicional. logicRules es la lista de reglas léxicas.

```
lexer :: String -> Either LexError [Token]
lexer = runLexer logicRules
```

Tenemos una función que puede leer `"(p ->q) & !p"` y nos devolverá

`[TLParen, TVar "p", TImplies, TVar "q", TRParen, TAnd, TNot, TVar "p"]`

Puede parecer más confuso a nuestros ojos. Pero en cambio está más cerca de que lo entiende el ordenador.

4.2. La gramática LL(1) para la lógica, el AST y el analizador sintáctico

Pasemos al segundo estadio del procedimiento que muestra la figura 2. Es el momento de dotar de sintaxis a nuestro lenguaje. Pensemos que el lexer puede aceptar sin problema cadenas como `"(p ->q) & !p"`, así como `"p q) & p! ("`. Es claro que la primera está bien formada, mientras que la segunda no pertenece al lenguaje que queremos definir.

Empecemos entonces introduciendo la noción de fórmula que se usa para definir el lenguaje de la lógica proposicional. Para construir una fórmula, tenemos dos reglas de formación **F1** y **F2** (Manzano, Hodkinson y Huertas).

F1. Las letras proposicionales (es decir, las variables) son fórmulas.

F2. Si A y B son fórmulas, también lo son: $\neg A$, $A \wedge B$, $A \vee B$, $A \rightarrow B$, $A \leftrightarrow B$.

Fijémonos en que el concepto de **fórmula** es fundamental para nuestro lenguaje formal. De hecho, las cadenas de nuestro lenguaje van a ser precisamente fórmulas. Esto se relaciona directamente con las dos estructuras que hay que tener en cuenta para construir nuestro parser: la gramática y los AST (*Abstract Syntax Tree*).

Comencemos mirando la traducción natural a AST. En el código 8 se ve cómo se captura puramente el significado, pues el árbol consiste en una fórmula donde, o bien es la variable `Var String` (**F1**), o bien es un nodo símbolo (pueden ser nodos `And`, `Or`, `Not`, `Implies` o `Iff`) del que se desprenden hijos o subfórmulas (**F2**).

Listing 8: El AST tiene o bien una hoja Var, o un nodo símbolo junto a una o dos subfórmulas.

```
data Formula
    = And Formula Formula
    | Or Formula Formula
    | Not Formula
    | Implies Formula Formula
    | Iff Formula Formula
    | Var String
```

Fijémonos en el grafo de la figura 3 que nos ilustra la jerarquía semántica. Es decir, nuestra fórmula '$q \wedge \neg p$' va a requerir evaluar primero las variables q y p, luego la subfórmula $\neg p$, y finalmente, la conjunción que reúne a q y $\neg p$. En particular, nuestro elemento `Formula` solo reúne elementos con

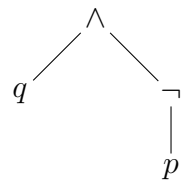

Figura 3: AST para la fórmula '$q \wedge \neg p$', representada en Haskell como `And (Var "q") (Not (Var "p"))`.

significado semántico. Los paréntesis, por ejemplo, solo hacen un trabajo de separadores. Es solo una tarea sintáctica la que tienen.

Con esto ya tenemos la construcción que queremos que nos genere el analizador. Pero nos falta cómo llegar a ella. La conversión de '$q \wedge \neg p$' a `And (Var "q") (Not (Var "p"))` nos ha parecido bastante directa. Pero fijémonos en un par de aspectos. En primer lugar, hemos dicho que tenemos los paréntesis para separar fórmulas. ¿Cómo los detectamos? Y, en caso de no tener paréntesis, ¿cuál signo tiene preferencia sobre cuál? Esta segunda problemática nos la muestra la figura 4: Como no tenemos definida la precedencia de los operadores, podemos generar el árbol de la izquierda, al que se otorga mayor precedencia a la implicación ($\rightarrow$), interpretándose como '$(a \rightarrow b) \wedge c$', y a su vez el de la derecha, donde la precedencia favorece a la conjunción ($\wedge$), interpretándose como '$a \rightarrow (b \wedge c)$'.

Figura 4: Dos posibles árboles de sintaxis abstracta (AST) para la misma cadena $a \rightarrow b \wedge c$.

La convención estándar en lógica proposicional se resuelve mediante la jerarquización de la gramática. Es, por lo tanto, necesario tener una gramática adecuada para la formación de las fórmulas. Tengamos en cuenta primero la precedencia clásica de los operadores:

$$\text{`}\neg\text{'} \longrightarrow \text{`}\wedge\text{'} \longrightarrow \text{`}\vee\text{'} \longrightarrow \text{`}\rightarrow\text{'} \longrightarrow \text{`}\leftrightarrow\text{'}$$

Esto significa que, por ejemplo, en el caso de la figura 4, '$a \rightarrow b \wedge c$' generaría el segundo árbol. Sería equivalente a '$a \rightarrow (b \wedge c)$'. La gramática CFG que podemos definir va a generar de arriba a abajo según la precedencia. Esto significa que primero va a intentar generar el operador ($\leftrightarrow$), luego el ($\rightarrow$), y así hasta las variables. Definamos entonces $G = \langle V, T, P, S \rangle$ dónde:

- El conjunto de no terminales lo definimos como $V = \{I, Imp, O, A, N, Atom\}$.
- El conjunto de terminales son los tokens definidos en el apartado anterior.
- El símbolo inicial es I: $S = I$.
- Las producciones de P son las siguientes:
  - $I \longrightarrow I \leftrightarrow I \mid Imp$

- $Imp \longrightarrow Imp \rightarrow Imp \mid O$
- $O \longrightarrow O \vee O \mid A$
- $A \longrightarrow A \wedge A \mid N$
- $N \longrightarrow \neg N \mid Atom$
- $Atom \longrightarrow (\ I\ ) \mid \text{var}$

Hagamos algunas observaciones a nuestra gramática G. En primer lugar, es cierto que nos sirve para generar las fórmulas de nuestro lenguaje, imponiendo la precedencia que hemos definido. Pero ahora mismo esta CFG no es una LL(1). Si bien cumple con el *Left Factoring*, tenemos un problema con la *Left Recursion*. Sin embargo, esta gramática se puede ajustar para que sea una LL(1) con la eliminación de la *Left Recursion* (Khan). Definimos $G' = \langle V', T', P', S' \rangle$ dónde:

- Ampliamos el conjunto de no terminales para poder evitar una transición por la izquierda hacia el mismo no terminal. Lo definimos como $V' = \{I, I', Imp, Imp', O, O', A, A', N, Atom\}$.
- El conjunto de terminales es el mismo: $T' = T = \{`(', `)', `\neg', `\wedge', `\vee', `\rightarrow', `\leftrightarrow', `\text{var}'\}$.
- El símbolo inicial también es el mismo: $S' = S = I$.
- Las producciones de P' fragmentan las de P para evitar la recursión por la izquierda.

Observemos que esta nueva definición nos permite quitar la recursión por la izquierda, derivando la tarea a un nuevo no terminal. G' es, por lo tanto, la gramática sobre la que basaremos nuestro analizador sintáctico.

Nuestra función principal va a ser `parseLogic`, que tomará una cadena de tokens, y nos devolverá, o bien un mensaje de error, o bien un AST, que hemos definido como `Formula`. Así que nuestra función de análisis sintáctico va a utilizar un `Parser` principal, que se dedique a inicializar el análisis asegurándose de que acaba. El parser en cuestión, analizará una cadena de tipo `Token` y devolverá elementos de tipo `Formula` (el AST). Podemos ver este parser definido en el código 9.

Listing 9: El Parser principal inicializa al análisis llamando al Parser que corresponde al no terminal I, y se asegura de que termine usando la funció `eof`.

```
parseRunning :: Parser Token Formula
parseRunning = do
    f <- parseI
    eof
    return f
```

Vemos que hemos usado la clase `Monad` para encadenar el resultado del `parseI`. A su vez, vemos en el código 10 que `parseI` hace correr el `parseImp`, que se encargará de buscar si hay la fórmula con ($\rightarrow$). Luego, si ha llegado a encontrar un hijo (`parseImp` ha encontrado una subfórmula), tratará de aplicar `parseI'`, que se encargará de verificar que realmente estemos en disposición de una fórmula con ($\leftrightarrow$).

Listing 10: Los parsers que corresponden al los no terminales I y I'. Se encargan de buscar si hay una fórmula con un ($\leftrightarrow$).

```
parseI :: Parser Token Formula
parseI = do
    leftSide <- parseImp
    parseI' leftSide

parseI' :: Formula -> Parser Token Formula
parseI' leftAST =
    (do
        _ <- satisfy (== TIff)
        rightSide <- parseImp
        let newAST = Iff leftAST rightSide
        parseI' newAST
    ) <|> return leftAST
```

Por otra parte, como hemos comentado, el `parseI'` verifica que haya el token `TIff` ($\leftrightarrow$). Esta verificación la podemos hacer con la función `satisfy` que hemos definido antes. En este punto tendríamos el árbol `Iff Formula1 Formula2`, dónde `Formula1` la hemos calculada desde el `parseI` (es la subfórmula que llamamos `leftSide`), y `Formula2` la vamos a calcular aplicando de nuevo el `parseImp`. Como puede ser que este árbol sea en realidad una subfórmula de otro nodo ($\leftrightarrow$), volvemos a aplicar `parseI'`. En el caso de que no haya un token ($\leftrightarrow$), podemos hacer *backtracking* aplicando el operador `<|>`. Esto detiene esta búsqueda de ($\leftrightarrow$) sin propagar el error.

Este procedimiento se traslada a los demás parsers para cada uno de los símbolos no terminales. De esta manera hacemos una construcción análoga con los parsers `parseImp` y `parseImp'`, `parseO` y `parseO'`, y `parseA` y `parseA'`. Finalmente, los `parseN` y `ParseAtom` implementan una lógica sencilla: El primero trata de buscar un símbolo ($\neg$), y si no, hace *backtracking*. El último trata de buscar una fórmula entre paréntesis, en la que trata de hacer el recorrido de arriba a abajo de nuevo si ha encontrado el primer paréntesis, y si no busca la variable que debe haber en la hoja del árbol que está explorando.

Pensemos que cuando actúa el `parseAtom`, significa que no hay más nodos disponibles sobre los que encapsular el token siguiente (a no ser que haya paréntesis y, por lo tanto, sí pueda haber un árbol formado por cualquiera de nuestros nodos). Esto significa que lo único que puede pasar es que el próximo token sea una variable. Usamos `parseVar` para guardar el nombre de la variable (recordemos que hay infinitas representaciones, ya que la hoja del AST es `Var String`).

Con ello, ya tenemos la construcción entera de nuestro analizador sintáctico. Al pasarle a la función `parseLogic` la lista de tokens

```
[TLParen, TVar "p", TImplies, TVar "q", TRParen, TAnd, TNot, TVar "p"]
```

obtendremos el AST, de tipo `Formula`,

```
And (Implies (Var "p") (Var "q")) (Not (Var "p"))
```

que en nuestra notación habitual se traduce como $(p \to q) \wedge (\neg p)$.

4.3. Evaluación semántica

Para terminar el procedimiento de análisis de cadenas de la lógica proposicional, podemos aprovechar el AST (*Abstract Syntax Tree*) generado por el parser para dar una interpretación a las fórmulas generadas (recordemos que el AST es `Formula`).

El lenguaje de las fórmulas válidas, así como el de las satisfacibles de la lógica proposicional, es decidible (Manzano, Hodkinson y Huertas). Esto significa que hay un método algorítmico para decidir la veracidad de cualquier fórmula establecida una interpretación. El método más conocido es el de las tablas de verdad, que nos permiten determinar el valor de verdad de una fórmula dados sus componentes. Determinar si una fórmula es una tautología no depende de una única interpretación de sus variables. De hecho, lo que caracteriza a una fórmula válida es que su valor de verdad es `True` para todas las posibles interpretaciones de sus variables. De ello se deriva que toda tautología es, en particular, satisfacible (pues ser satisfacible solo exige ser verdad en al menos una interpretación).

Al tener un AST de tipo `Formula`, podemos establecer un método recursivo de decisión (Alonso Jiménez, Tema 9) que nos busque los posibles valores de verdad de una fórmula, dando solamente las reglas de los operadores (las tablas de verdad para cada operador). Así, podemos definir una función `esTautologia` que, al darle un AST tipo `Formula`, nos devuelva el valor de verdad correspondiente a verificar si es una tautología.

La mecánica de la función consiste en extraer el conjunto de variables del AST y generar todos los posibles entornos de verdad: es decir, todas las combinaciones de asignaciones de valores de verdad para dichas variables. Si el AST tiene n variables distintas, habrá 2^n entornos posibles. La fórmula es una tautología si y solo si se cumple que el valor de verdad generado para cada entorno es True. Este proceso pone de manifiesto la naturaleza decidible de la lógica proposicional.

Finalmente, ensamblamos los tres bloques: el analizador léxico, el analizador sintáctico, y el evaluador de tautologías, para obtener una función

```
evaluateLogicTautologies :: String -> Maybe (Formula , Bool)
```

que al pasarle una cadena de caracteres, nos devuelva, o bien un error (con `Nothing`) o bien un par `(Formula , Bool)`. Es decir, el AST junto a su valor de verdad.

Así, al pasarle `"(p ->q) & !p"` nos devuelve

```
(And (Implies (Var "p") (Var "q")) (Not (Var "p")),False)
```

y al pasarle `"p | !p"` (principio del tercio excluido), nos devuelve

```
(Or (Var "p") (Not (Var "p")),True)
```

Este ha sido un ejemplo del abanico de posibilidades que nos ofrece el analizador sintáctico. Una vez tenemos el AST generado de una cadena, el ordenador "sabe" qué es una fórmula. Y, puesto que la lógica proposicional es decidible, podemos dar las asignaciones que queramos y emplear el método de las tablas de verdad para encontrar el valor de verdad de cualquier fórmula procesada por el parser.

5. Conclusiones

Con este artículo hemos podido comprobar la utilidad de los modelos de computación más elementales a través de una aplicación práctica de gran interés. La creación de herramientoas como son los analizadores de texto (**lexer** y **parser**), siguen teniendo a día de hoy una gran relevancia en la computación aplicada. Su creación no es un trabajo independiente del cuerpo principal del proyecto Pons Garcia, sino que materializa, en un mismo proceso continuado, los dos primeros niveles de la jerarquía de Chomsky: el tipo 3 para el análisis léxico y el tipo 2 para el análisis sintáctico. Este enlace constituye el puente más directo entre la teoría y la práctica que presenta el trabajo.

Además, este artículo conforma tan solo una primera muestra de lo que se puede hacer con estos modelos. Partiendo de las definiciones de los ADT (*Abstract Data Type*) que hemos dado en Haskell, tenemos por delante un mar de posibles herramientas para ele análisis de texto.

Referencias

Aho, Alfred V. et al. *Compilers: Principles, Techniques, and Tools* [**inlang**English]. Second. Boston, MA: Pearson Education / Addison Wesley, 2006. ISBN: 978-0321486813.

Alonso Jiménez, José A. *Informática (1º de Matemáticas)*. Material docente, Universidad de Sevilla, 2025. https://www.cs.us.es/~jalonso/cursos/i1m/.

Hopcroft, John E., Rajeev Motwani y Jeffrey D. Ullman. *Teoría de autómatas, lenguajes y computación*. 3.ª ed. Madrid, España: Addison-Wesley, 2007. ISBN: 978-84-7829-088-8.

Hutton, Graham y Erik Meijer. «Monadic parsing in Haskell». *Journal of Functional Programming* 8, n.º 4 (1998): 437-444. https://doi.org/10.1017/S0956796898003050.

Khan, Rosina S. *The Dummies' Guide to Compiler Design*. E-book en línea (https://rosinaskhan.weebly.com/). Disponible en Free-eBooks.net e Internet Archive, 2018.

Manzano, María, Ian Hodkinson y Antonia Huertas. *Lógica para principiantes*. Coordinadora: María Manzano. Universidad de Salamanca. Proyecto ARACNE, 2000. http://aracne.usal.es.

Pons Garcia, Martí. «Formalización y verificación de lenguajes formales mediante teoría de autómatas». Tutor: Fernando Soler Toscano. Máster en Lógica y Filosofía de la Ciencia. Tesis de mtría., Interuniversitario (Universidad de Granada, Universidad de Salamanca, Universidad de Valladolid), mar. de 2026.